

Etap	1	2	3	4	5	Suma
Punkty	2	5	6	4	3	20
Wynik						

L4: Wybory cesarskie

Jest rok 1519. Wraz ze śmiercią cesarza Świętego Cesarstwa Rzymskiego Maksymiliana I nadszedł czas wyboru nowego cesarza. Jest trzech kandydatów:

1. Franciszek I, król Francji,
2. Karol V, arcyksiążę Austrii i król Hiszpanii,
3. Henryk VIII, król Anglii.

W wyborze udział bierze siedmiu elektorów z różnych państw członkowskich Świętego Cesarstwa Rzymskiego:

1. Moguncja,
2. Trewir,
3. Kolonia,
4. Czechy,
5. Palatynat,
6. Saksonia,
7. Brandenburgia.

W tym roku, aby ułatwić proces elekcji, postanowiono utworzyć serwer TCP przyjmujący i liczący głosy. Twoim zadaniem jest napisanie tego serwera. Każdy elektor będzie się z nim łączył, oddawał swój głos, ewentualnie zmieniał swój głos, a następnie rozłączał się. Możesz użyć **netcat** jako programu klienta.

Stages:

1. 2 p. Zaimplementuj przyjmowanie pojedynczego połączenia do serwera. Serwer przyjmuje jeden argument, numer portu. Przykładowe uruchomienie serwera:

```
./sop-hre 8888
```

Po uruchomieniu serwer oczekuje na pojedyncze połączenie TCP. Po nawiązaniu połączenia serwer wypisuje „Klient połączony”, zamyka połączenie i kończy działanie.

2. 5 p. Zaimplementuj połączenia od wielu klientów.

Po połączeniu klienta wyślij mu wiadomość powitalną **Welcome, elector!**. Następnie wypisz każdą wiadomość, którą klient przesyła do serwera, na **stdout**. Użyj **epoll** do zaimplementowania tego etapu (lub, alternatywnie, **ppoll** lub **pselect**).

Pamiętaj o poprawnym obsłudze rozłączających się klientów.

Ciąg dalszy zadania na następnej stronie...

3. 6 p. Zaimplementuj proces głosowania i przechowywanie listy połączonych elektorów. Gdy nowy klient się połączy, czekaj na pojedynczą cyfrę (w zakresie $[1, 7]$). Ta cyfra to numer elektora łączącego się z serwerem. Jeśli od klienta otrzymano inny znak, zakończ jego połączenie. Podczas oczekiwania na wiadomość identyfikującą, inni klienci powinni nadal móc się łączyć z serwerem. Jeśli inny klient identyfikujący się jako elektor E próbuje połączyć się z serwerem, powinien zostać odłączony. Wiadomość wysłana do klienta powinna zostać wysłana po identyfikacji i powinna zawierać `Welcome, elector of X!`, gdzie X to państwo członkowskie, z którego pochodzi elektor.

Po otrzymaniu wiadomości identyfikującej, połączony klient powinien móc oddawać głosy w wyborach, przysyłając liczbę z zakresu $[1, 3]$, oznaczającą preferowanego kandydata. Inne znaki powinny być ignorowane. Dowolny elektor może zmienić zdanie i zagłosować wielokrotnie. Kolejne głosy nadpisują poprzednie.

Poprawnie obsługuj rozłączających się klientów z serwera, w tym rozłączających się i ponownie łączących się elektorów!

4. 4 p. Zaimplementuj dodatkowy wątek wysyłający wiadomości UDP. Program powinien teraz akceptować łącznie dwa argumenty: port serwera TCP i port klienta UDP. Przykładowe wykonanie serwera:

```
./sop-hre 8888 9999
```

Cała funkcjonalność z poprzednich etapów nadal powinna działać. Po uruchomieniu serwera utwórz dodatkowy wątek z klientem UDP wysyłającym co sekundę wiadomość z obecnym wynikiem elekcji na podany port na `localhost`. Pamiętaj aby zabezpieczyć wyniki elekcji mutexem lub w inny sposób zabezpieczyć się przed data race.

5. 3 p. Obsłuż sygnał `SIGINT`. Po jego otrzymaniu, zakończ wszystkie aktywne połączenia z elektorami i zwolnij wszystkie zasoby, włączając w to wątek UDP. Policzb i wydrukuj głosy dla każdego kandydata.